BERT-Based Requirement Classification for Agile Backlog Management

[Author Name]
[Department, Institution]
[City, Country] · [e-mail]

***Abstract*—Product backlogs mix feature requests, quality constraints, defects and chores, and few teams label them consistently. Quality requirements buried in this mix tend to surface late, when they are costly to fix. This paper asks whether BERT-family encoders can classify backlog items reliably enough to support refinement and sprint planning. We synthesise 35 verified sources on classical, deep-learning, transformer and zero-shot classifiers, and propose a two-level framework whose confidence gate sends uncertain items to the Product Owner rather than labelling them silently. In our own replication on the PROMISE NFR dataset (625 requirements), TF-IDF baselines and a partially frozen, CPU-trained DistilBERT shared identical stratified 10-fold splits. DistilBERT matched but did not beat a linear SVM on functional versus non-functional classification (macro-F1 0.901 vs. 0.916; McNemar p = 0.35), yet it clearly outperformed every baseline on the four most frequent quality classes (0.919 vs. 0.832; p < 0.001). Published BERT scores on PROMISE reach 0.94–0.99 F1, but, like ours, they rest on one small, dated corpus and mostly on random rather than project-level splits. We conclude that BERT classifiers suit refinement assistance better than autonomous labelling, and we list the evidence still missing for real backlogs.**

***Index Terms*—requirements classification, BERT, DistilBERT, non-functional requirements, agile, product backlog, user stories, PROMISE NFR**

I. Introduction

In Scrum the Product Backlog is the single, ordered source of work for a team, and the Product Owner is accountable for keeping it in order (Schwaber & Sutherland, 2020). In practice the backlog is messier than that one sentence suggests. In a grounded-theory study of eight projects and 56 interviews at Pivotal, Sedano et al. (2019) found that the backlog acts as an informal model of the work still to be done, and that it serves as a boundary object shared by the different roles on a team rather than as a requirements specification. Most of its items are written as user stories; Lucassen et al. (2016b) surveyed 182 practitioners and interviewed 21 more, and found the Connextra template ("As a ⟨role⟩, I want ⟨goal⟩, so that ⟨benefit⟩") to be the dominant format.

Consider a typical story: "As a user, I want to reset my password through an e-mailed link so that I can regain access to my account." On the surface it is functional. Read carefully, it also carries security obligations: the link must expire, it must work only once, and the response must not reveal whether an e-mail address is registered. If the team tags the story only as a feature, those constraints are easily missed until a penetration test or an incident exposes them. Dalpiaz et al. (2019) formalised this observation by annotating requirements with separate functional and quality *aspects*, so that a single item can carry both. Van Can and Dalpiaz (2025) analysed 1,636 backlog items from 14 projects and found that requirement-related content is labelled inconsistently and that one item often contains several requirements at different levels of granularity.

Automatic classification of requirements text has a long history. It started with indicator-term classifiers trained on student specifications (Cleland-Huang, Settimi, et al., 2007) and moved through feature-engineered support vector machines (Kurtanović & Maalej, 2017) and convolutional or recurrent networks (Navarro-Almanza et al., 2017; Rahman et al., 2019). Since BERT appeared (Devlin et al., 2019), fine-tuned transformers have become the default, and NoRBERT showed that transfer learning helps most on projects not seen during training (Hey et al., 2020a). Yet three gaps remain. First, almost every result comes from the PROMISE NFR corpus, a set of 625 "shall" statements written for 15 student projects in the mid-2000s (Cleland-Huang, Mazrouee, et al., 2007), not from agile backlogs. Second, scores are reported under very different protocols, so a 0.99 in one paper and a 0.87 in another cannot be compared directly. Third, few studies say what a team should *do* with a predicted label, or how to handle the predictions a model is unsure about.

This paper addresses those gaps with four contributions:

1) a critical synthesis of 35 sources, with the reported results collected in a single comparison table (Table I) that keeps each study's dataset and protocol visible;
2) a two-level classification framework (Figs. 1–3) that connects predicted labels to backlog refinement, the Definition of Done and sprint planning, and that routes uncertain items to a human through an explicit confidence gate;
3) a small replication that we ran ourselves on PROMISE NFR, comparing DistilBERT with three TF-IDF baselines on identical folds, with every hyper-parameter and limitation stated;
4) a discussion of deployment challenges and threats to validity, summarised in Table IV.

Section II reviews related work, Section III presents the framework, Section IV reports our experiment and the literature results separately, Section V discusses challenges and validity, and Section VI concludes.

II. Related Work

A. Datasets and Label Schemes

The PROMISE NFR dataset was created by Cleland-Huang, Settimi, et al. (2007) and later shared through the PROMISE repository (Cleland-Huang, Mazrouee, et al., 2007). It

contains 255 functional requirements and 370 non-functional ones, the latter spread over eleven quality classes ranging from usability (67 items) to portability (one item). Lima et al. (2019) enlarged it to 969 requirements (444 functional, 525 non-functional) in PROMISE_exp, but kept the same style of text. Dalpiaz et al. (2019) relabelled more than 1,500 requirements from PROMISE and seven industrial documents with independent functional and quality aspects. That relabelling matters. It shows that the binary FR/NFR split, which most papers take for granted, forces an annotator to discard information whenever a requirement constrains behaviour and quality at the same time.

B. Classical and Deep-Learning Classifiers

Kurtanović and Maalej (2017) combined lexical and syntactic features with a support vector machine and used under- and over-sampling to counter class imbalance. They reported precision and recall of up to about 92% for the FR/NFR distinction on PROMISE NFR, with security and performance among the best-recognised quality classes. Dalpiaz et al. (2019) replaced the large feature space with 17 interpretable features derived from dependency parsing. Their classifier fitted the training data slightly less well than a reconstruction of the Kurtanović and Maalej model, but it generalised better to other datasets and its decisions could be explained. To us this is the more useful result for practice, because a Product Owner is more likely to trust a tag whose cause can be shown.

Neural models followed. Winkler and Vogelsang (2016) used a convolutional network to separate requirements from explanatory text in industrial specifications and reached a precision of 0.73 and a recall of 0.89. Navarro-Almanza et al. (2017) applied a CNN with word embeddings to the twelve PROMISE classes and obtained a precision of 0.80, a recall of 0.785 and an F1 of 0.77. Rahman et al. (2019) compared recurrent variants on PROMISE NFR and reported that an LSTM reached a precision of 0.973, a recall of 0.967 and an F1 of 0.966, ahead of GRU and CNN models. Rahimi et al. (2021) stacked LSTM, BiLSTM, GRU and CNN models into an ensemble and reported 92.56% accuracy for one-phase classification. Their two-phase variant reached 95.7% for the binary step and 93.4% for the multi-class step. Two reviews put this stream in context: Binkhonain and Zhao (2019) catalogued machine-learning methods for identifying non-functional requirements, and Zhao et al. (2022) mapped the wider NLP-for-RE field and documented the gap between research prototypes and tools that practitioners actually use.

A pattern runs through these numbers. The highest scores tend to come from the smallest evaluation sets and the least clearly described splits. An F1 of 0.966 on roughly 370 short sentences is plausible under random cross-validation, where requirements from the same project, often with near-identical wording, fall on both sides of a split. It says little about how the same model would label a new team's backlog.

C. BERT and Its Variants in Requirements Engineering

BERT pre-trains a bidirectional transformer encoder with masked-language-model and next-sentence objectives, and is then adapted to a downstream task by adding a single output layer and fine-tuning all weights (Devlin et al., 2019). RoBERTa keeps the architecture but trains longer on more data, drops the next-sentence objective and masks tokens dynamically (Liu et al., 2019). DistilBERT applies knowledge distillation to obtain a model about 40% smaller and 60% faster that keeps 97% of BERT's language-understanding performance (Sanh et al., 2019), which makes it a realistic choice for a team without GPUs. Sun et al. (2019) studied fine-tuning choices such as layer selection, learning rates and further pre-training, and their recommendations inform the settings in Section IV.

NoRBERT was the first systematic use of BERT for requirements classification (Hey et al., 2020a). On PROMISE NFR it reached an F1 of up to 94% for FR/NFR, an average F1 of 87% on the most frequent quality classes, and gains of up to 15 percentage points over earlier work when tested on unseen projects. The authors later reported a bug that made their project-specific folds not strictly project-specific; corrected scores were about five points lower, although the advantage on unseen projects held (Hey et al., 2020b). We regard this correction as one of the most instructive episodes in the field. It shows how easily leakage between folds can inflate results, and it was disclosed openly.

Later studies widened the scope. Kici et al. (2021) found that DistilBERT outperformed LSTM and BiLSTM baselines on several requirements datasets. Luo et al. (2022) reformulated classification as prompt-based cloze filling (PRCBERT) and, with self-learning, reached zero-shot F1 scores of 53.27% on NFR-Review and 72.96% on NFR-SO. Domain-specific work includes aeroBERT, which classified 310 aerospace requirements with an average F1 of 0.83 (Tikayat Ray et al., 2023), and BERTurk fine-tuned on newly created Turkish requirements, which reached an F-score of about 95% (Yucalar, 2023). Alsanoosy (2026) assembled 4,908 requirements and reported a binary F1 of 86.93% for a BERT+LSTM hybrid, while Eltahier et al. (2025) reported an F1 of 0.99 on PROMISE and 0.97 on the FR_NFR dataset after varying model components and training-set size. Read side by side, these results show that the dataset and the split protocol explain more of the variation than the choice of encoder. A score of 0.99 on PROMISE and 0.87 on a corpus eight times larger do not show that one method is better than the other.

D. Zero-Shot, Few-Shot and Large Language Models

Labelling is expensive, so learning from few or no labelled examples is attractive. Alhoshan et al. (2023) tested zero-shot classification with pre-trained sentence embeddings and label descriptions. Their best zero-shot F1 was 0.66 for functional and 0.65 for non-functional requirements, against 0.93 for the supervised Kurtanović–Maalej model on FR and 0.93 for NoRBERT on NFR. The gap is large but so is the saving: no labelled data at all. SetFit contrastively fine-tunes a sentence transformer on a handful of pairs; with only eight labelled examples per class it was competitive with fully fine-tuned RoBERTa-Large on a sentiment benchmark (Tunstall et al., 2022). To our knowledge it has not yet been evaluated on

PROMISE, which makes it a natural next step. The broader shift towards large language models is documented in the review by Hou et al. (2024). For backlogs specifically, van Can and Dalpiaz (2025) found that fine-tuned encoder-only models were better suited than prompted generative models to locating requirements in backlog items. This is a useful corrective to the assumption that the largest model is always the right one.

E. Agile Backlog Support

Other work automates neighbouring backlog tasks. Lucassen et al. (2016a) proposed the Quality User Story framework, with 13 criteria, and the AQUSA tool for detecting defects in story text. Choetkiertikul et al. (2019) estimated story points with an LSTM and a recurrent highway network trained on 23,313 issues from 16 open-source projects. Mejía-Domínguez et al. (2026) built LOONA, an LLM assistant for the quality assurance of Spanish user stories, and reported a κ of 0.76 in an evaluation on 100 stories. Prioritisation is the step that ultimately consumes any label, and Hujainah et al. (2018) reviewed the techniques, stakeholders and open challenges of requirements prioritisation. None of these studies feeds requirement-type labels into prioritisation or refinement, and that is the link our framework is meant to provide.

TABLE I
Reported results from the literature (not our experiments). Scores are copied as published; datasets and protocols differ, so rows are not directly comparable.

Study	Data	Approach	Result as reported by the authors	Our remark
(Kurtanović & Maalej, 2017)	PROMISE NFR	SVM, lexical + syntactic features, sampling	FR/NFR P and R up to ≈92%	Strong classical baseline
(Dalpiaz et al., 2019)	PROMISE + 7 industrial sets (>1,500 items), F/Q aspects	Dependency-parse features (17)	Generalises better than K&M reconstruction	Interpretable; multi-aspect labels
(Winkler & Vogelsang, 2016)	Industrial specifications	CNN (requirement vs. information)	P 0.73, R 0.89	Different task
(Navarro-Almanza et al., 2017)	PROMISE, 12 classes	CNN + word embeddings	P 0.80, R 0.785, F1 0.77	Includes rare classes
(Rahman et al., 2019)	PROMISE NFR	LSTM / GRU / CNN	LSTM: P 0.973, R 0.967, F1 0.966	Split protocol limits comparison
(Rahimi et al., 2021)	PROMISE	Ensemble of DL models	Acc. 95.7% (binary), 93.4% (multi-class), two-phase	Accuracy only
(Hey et al., 2020a)	PROMISE NFR	NoRBERT (BERT fine-tuning)	FR/NFR F1 up to 94%; frequent NFR avg. F1 87%	Corrected folds ≈ 5 pp lower (Hey et al., 2020b)
(Kici et al., 2021)	PROMISE NFR and others	DistilBERT	Outperformed LSTM and BiLSTM	Light model
(Luo et al., 2022)	NFR-Review, NFR-SO	PRCBERT prompt learning	Zero-shot F1 53.27% / 72.96%	Non-SRS text
(Alhoshan et al., 2023)	PROMISE NFR and others	Zero-shot embeddings	Best F1 0.66 (FR), 0.65 (NFR)	No labels needed
(Tikayat Ray et al., 2023)	310 aerospace requirements	aeroBERT-Classifier	Average F1 0.83	Domain-specific
(Yucalar, 2023)	New Turkish requirements	BERTurk	F-score ≈ 95%	Non-English
(Alsanoosy, 2026)	4,908 requirements	BERT + LSTM	Binary F1 86.93%	Largest corpus here
(Eltahier et al., 2025)	PROMISE; FR_NFR	BERT fine-tuning variants	F1 0.99 (PROMISE), 0.97 (FR_NFR)	Near ceiling; needs scrutiny
(van Can & Dalpiaz, 2025)	1,636 backlog items, 14 projects	Encoders vs. generative LLMs	Encoder-only models most suitable	Closest to backlogs

III. Methodology and Proposed Framework

A. Research Approach

The study combines a structured, but not systematic, literature review with a small experiment. In October 2026 we searched IEEE Xplore, the ACM Digital Library, SpringerLink, ScienceDirect, MDPI and arXiv through web search. We used combinations of *requirements classification*, *non-functional*, *BERT*, *user story*, *product backlog*, *zero-shot* and *class imbalance*. We screened around 60 candidate records and kept 35. A source was kept if it reported a classifier for natural-language requirements, contributed a dataset or model that such work depends on, or described agile backlog practice. Every DOI was checked against Crossref, DataCite or arXiv metadata. Because we did not follow a registered protocol, did not double-screen, and did not snowball exhaustively, we do not claim the coverage of a systematic literature review.

B. Framework Overview

Fig. 1 shows the proposed framework. Items enter from the team's tracker (for example Jira), from a story-writing workshop or from user feedback. A pre-processing step normalises each item: it splits a Connextra story into role, goal and benefit, keeps the “so that” clause because it often signals the quality concern, separates acceptance criteria, and removes personal names. Classification then proceeds in two levels. Level 1 decides whether an item contains a quality concern: FR versus NFR, or F+Q when both aspects are present, as proposed by Dalpiaz et al. (2019). Level 2 assigns the quality class, such as security (SE), performance (PE), usability (US) or operability (O), and is run only for items that Level 1 flags.

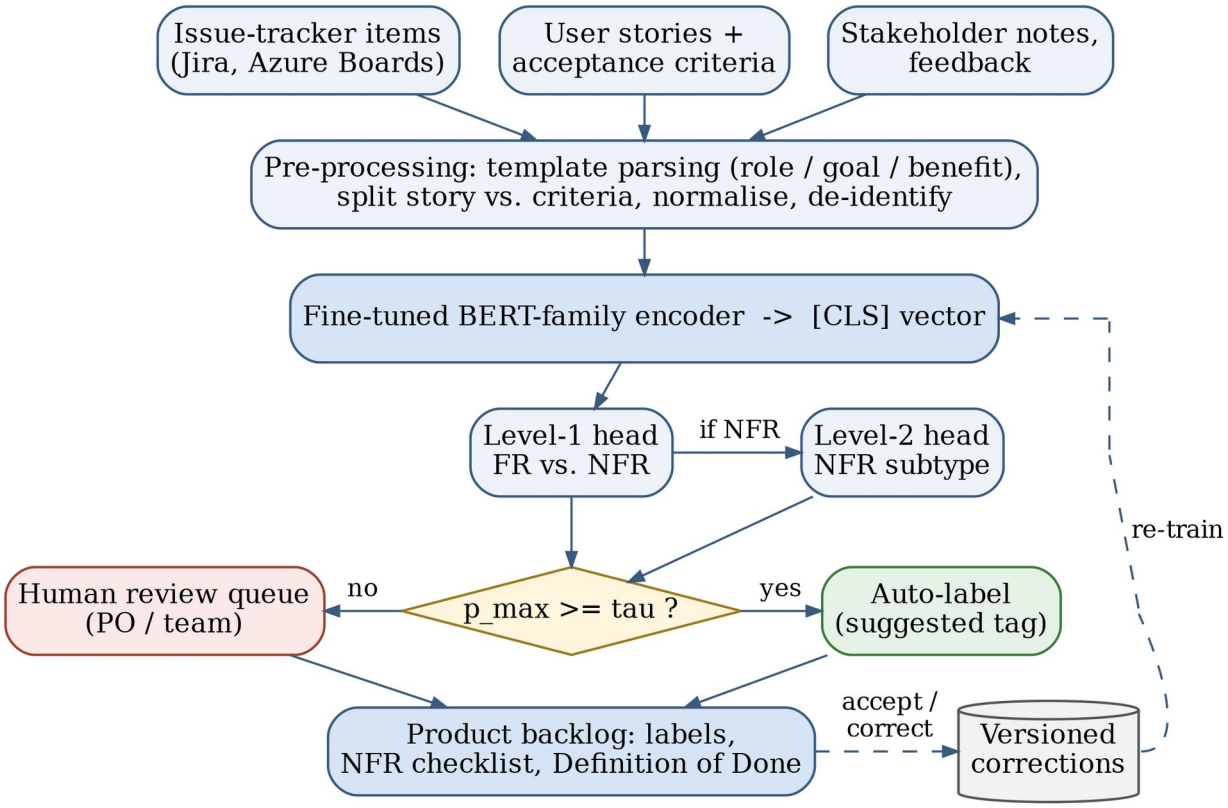


Fig. 1. Proposed framework. Items are normalised, encoded by a fine-tuned BERT-family model and classified at two levels. A confidence gate decides whether a label is applied automatically or sent to the Product Owner, whose corrections feed periodic re-training.

C. Encoder and Training Objective

Any BERT-family encoder fits the framework. We use DistilBERT in Section IV because it runs on a laptop CPU. Fig. 2 traces the running example through the model. The text is tokenised into WordPiece units, the final hidden state of the [CLS] token passes through a small classification head, and a softmax gives a probability for each of the K labels. PROMISE NFR is imbalanced: usability has 67 examples and fault tolerance has 10. The training loss therefore weights each class by its inverse frequency, which is a standard algorithm-level remedy for imbalance (Johnson & Khoshgoftaar, 2019):

$$\mathcal{L} = -(1/N)\,\Sigma_{i=1}^{N}\, w_{y_i} \log p_\theta(y_i \mid x_i), \qquad w_c = N / (K \cdot N_c) \qquad (1)$$

Here N is the number of training items, N_c the number of items in class c and K the number of classes. Re-sampling, as used by Kurtanović and Maalej (2017), is an alternative. We prefer weighting because it leaves the data untouched and adds no duplicated sentences that could appear in both training and test folds.

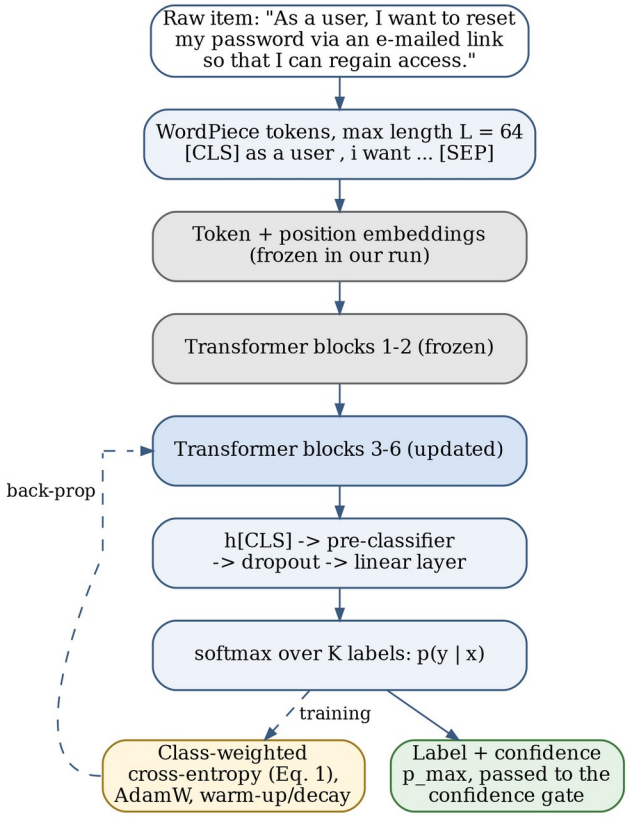


Fig. 2. Fine-tuning pipeline for one backlog item. Grey blocks were frozen in our CPU experiment; the dashed path is used only during training.

D. Confidence Gate

A classifier that is wrong 8% of the time and gives no warning will gradually erode the team's trust in the backlog. The framework therefore applies a label automatically only when the model is confident:

$$\text{route}(x) = \text{auto-label } \hat{y} \text{ if } \max_c p_\theta(c \mid x) \geq \tau, \text{ else PO review} \quad (2)$$

The threshold τ is set on held-out data so that the automatically applied labels reach a precision the team agrees on, for example 0.95. Softmax probabilities from fine-tuned transformers are often over-confident, so a team should check calibration before relying on τ. Items below the threshold appear in the refinement queue with the two most likely labels shown, which turns a silent error into a one-click decision.

E. Integration with the Backlog Workflow

Fig. 3 places the classifier in the Scrum cycle. When an item is created or edited, the tracker sends it to the service, which writes back a label and a confidence value. During refinement the team reviews the low-confidence queue and also checks the high-confidence quality labels, because these carry consequences. A security label, for instance, adds the team's security checklist to the item's acceptance criteria. A performance label prompts the team to state a measurable target, such as "search results appear within 2 s for 95% of queries." Usability items get an accessibility check. Before sprint planning, the Product Owner can filter the ordered backlog by label and see, for example, that the next three sprints contain no operability work. Ordering stays with the Product Owner (Schwaber & Sutherland, 2020); the classifier makes quality work visible but never reorders the backlog.

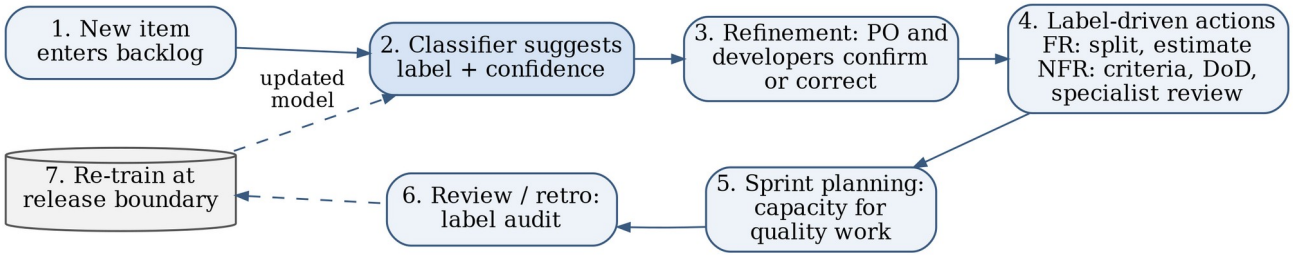


Fig. 3. Classifier-in-the-loop backlog workflow. Corrections made during refinement are logged and used for re-training at release boundaries.

F. Human Feedback and Re-Training

Every label the Product Owner confirms or changes is stored together with the item text, the model version and the confidence. The model is re-trained at release boundaries rather than continuously, so that its behaviour stays stable within a sprint. Two signals tell the team when re-training is due: the share of items routed to review rising above an agreed level, and the precision of automatic labels, measured on a small random audit sample, falling below the target.

IV. Experimental Setup and Results

This section has two parts. Section IV-A reports an experiment that *we ran ourselves*. Section IV-B summarises results *reported in the literature*, which we did not reproduce. The two are kept separate throughout, and no literature number appears in our results table.

A. Our Replication on PROMISE NFR

Data. We downloaded the PROMISE NFR dataset from its Zenodo record (Cleland-Huang, Mazrouee, et al., 2007) and parsed the ARFF file into 625 labelled requirements from 15 projects. Table II gives the class distribution, together with an agile-style example that we wrote for each class to show what the label means in a backlog. The PROMISE items themselves are "shall" statements, not user stories. We defined two tasks. Task A separates functional (255) from non-functional (370) requirements. Task B classifies the four most frequent quality classes: usability (67), security (66), operability (62) and performance (54), 249 items in total. The seven remaining quality classes have between 1 and 38 examples, too few for a 10-fold estimate, so we left them out.

TABLE II
PROMISE NFR class distribution (625 requirements, 15 projects) and illustrative backlog items.

Code	Class	n	Task	Illustrative agile item (written by us, not from PROMISE)
F	Functional	255	A	"As a user, I want to reset my password via an e-mailed link so that I can regain access." (F; also carries a security aspect)
US	Usability	67	A, B	"As a screen-reader user, I want every form field to have a label so that I can complete checkout."
SE	Security	66	A, B	"As an administrator, I want failed login attempts logged with time and IP so that I can spot attacks." (F+SE)
O	Operability	62	A, B	"As an operator, I want to change the backup schedule without a redeploy."
PE	Performance	54	A, B	"As a shopper, I want search results within 2 s so that I do not abandon the search."
LF	Look and feel	38	A	"As a marketer, I want pages to use the corporate colour palette."
A	Availability	21	A	"As a customer, I want the shop to be reachable 99.9% of the time."
SC	Scalability	21	A	"As the business, we want to serve 10,000 concurrent users at launch."
MN	Maintainability	17	A	"As a developer, I want tax rules in configuration rather than in code."
L	Legal	13	A	"As a data-protection officer, I want users to be able to export their personal data."
FT	Fault tolerance	10	A	"As a user, I want my cart preserved if the payment service fails."
PO	Portability	1	A	"As a user, I want the app to run on Android 12 and later."

Protocol. All models share the same stratified 10-fold splits (scikit-learn, seed 42). We pool the out-of-fold predictions and report per-class precision, recall and F1, their macro averages and accuracy. We used random rather than project-held-out folds. Section V explains why this flatters every model, including ours. We did no hyper-parameter search for any model: the baselines use common defaults and DistilBERT uses the settings below. Tuning on the same folds would have made the comparison look better without making it more honest.

Baselines. Each baseline uses TF-IDF features over word unigrams and bigrams, with sublinear term frequency and English stop-word removal, fitted inside each training fold. On top of these features we trained multinomial Naive Bayes (α = 0.1), a linear SVM (C = 1, class-balanced weights) and logistic regression (C = 10, class-balanced weights).

DistilBERT. We fine-tuned distilbert-base-uncased (Sanh et al., 2019) with the Hugging Face Transformers library. Settings: maximum length 64 WordPiece tokens (median item length is 21 tokens; 7 of 625 items were truncated), batch size 8, AdamW with learning rate 5×10^{-5} and weight decay 0.01, 10% linear warm-up followed by linear decay, gradient clipping at 1.0, five epochs, and the class-weighted loss of (1). The sandbox had two CPU cores and about 2 GB of RAM, so we froze the embedding layer and the first two of the six transformer blocks, which reduced optimiser memory. This departs from full fine-tuning (Devlin et al., 2019), and the numbers below should be read as a lower bound for what DistilBERT can do on this data. One fold took about 236 s to train for Task A and 88 s for Task B.

Results. Table III and Fig. 4 give the results. On Task A the four models finish within 1.5 points of each other. The linear SVM has the highest macro-F1 (0.916) and DistilBERT the lowest (0.901). DistilBERT recognises non-functional requirements well (recall 0.943) but misses more functional ones (recall 0.851; 38 of 255 were labelled NFR). An exact McNemar test on the pooled predictions finds no significant difference between DistilBERT and the SVM (24 items only DistilBERT got right, 32 only the SVM; p = 0.35). Task B tells a different story. DistilBERT reaches a macro-F1 of 0.919, against 0.832 for the SVM, 0.824 for logistic regression and 0.783 for Naive Bayes, and its F1 is at least 0.91 for every class. Here the McNemar tests are significant against all three baselines (30 versus 8 discordant items against the SVM; p < 0.001 in each case), and the differences remain significant after a Bonferroni correction for three comparisons.

We read the contrast as follows, although we did not test this explanation. In PROMISE the FR/NFR split is carried largely by surface cues, such as verbs of action versus adjectives of quality, and a bag of n-grams captures these well. Telling usability from operability or security is harder: words such as “user”, “system” and “shall” appear in every class, and each class has only 54–67 examples. This is the situation in which pre-trained representations are expected to help most. The errors also support the F+Q view of Dalpiaz et al. (2019). Several Task A “errors” are debatable. One item requires that all changes to a dispute case be “recorded in the case history”; PROMISE labels it NFR, the model predicted F, and it reads like our audit-log example in Table II. In Task B, a requirement to conform to the Americans with Disabilities Act is labelled usability but was predicted security, and arguably it is a legal requirement as well. Fold-level accuracy varied widely (DistilBERT on Task B: 0.84 to 1.00 over test folds of about 25 items), which is why we report pooled scores.

TABLE III
Our experiment: pooled 10-fold results on PROMISE NFR (Task A: FR vs. NFR, n = 625; Task B: US/SE/O/PE, n = 249). mF1 = macro-F1; best value per column in bold.

Model	A: mF1	A: Acc.	B: mF1	B: Acc.
TF-IDF + Naive Bayes	0.907	0.910	0.783	0.783
TF-IDF + LogReg	0.914	0.917	0.824	0.823
TF-IDF + Linear SVM	**0.916**	**0.918**	0.832	0.831
DistilBERT (2 blocks frozen)	0.901	0.906	**0.919**	**0.920**

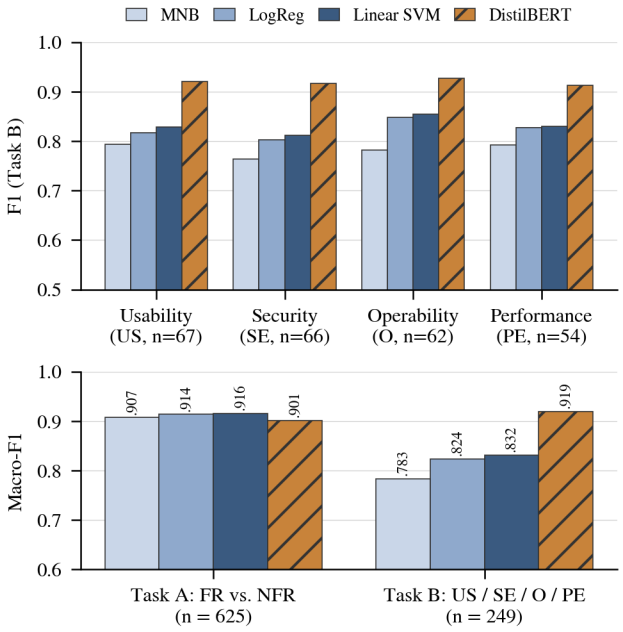


Fig. 4. Our experiment: per-class F1 on Task B (top) and macro-F1 on both tasks (bottom), pooled over the same 10 folds for every model. Note that the y-axes do not start at zero.

B. Reported Results from the Literature

Table I lists the results reported in the literature. These are the authors' numbers, not ours. On PROMISE NFR the published results bracket our own. For the FR/NFR decision, NoRBERT reports an F1 of up to 94% (Hey et al., 2020a) and Eltahier et al. report 0.99 (Eltahier et al., 2025), both above our 0.901. For the frequent quality classes, our DistilBERT macro-F1 of 0.919 is higher than the 87% average F1 that NoRBERT reports. We do not read this as DistilBERT beating NoRBERT. The protocols differ: we used random stratified folds over four classes, while NoRBERT's corrected project-level scores are about five points below its originally reported ones (Hey et al., 2020b), which indicates how much random or leaky splits can add. Model capacity differs in the opposite direction: NoRBERT fine-tuned full-size BERT models (base

and large) with all layers trainable, whereas we used a distilled model with a third of its blocks frozen. Our TF-IDF+SVM macro-F1 of 0.916 on Task A is close to the roughly 92% precision and recall reported by Kurtanović and Maalej (2017), so the classical baseline appears to be reasonably reproduced. The 0.99 reported by Eltahier et al. cannot be placed on the same protocol as ours from the information we used.

V. Discussion

A. What the Evidence Supports

Taken together, the sources support three claims and leave a fourth open. First, fine-tuned transformers are at least as good as feature-engineered models on PROMISE-style text and are better at transferring to unseen projects (Hey et al., 2020a; Hey et al., 2020b). Second, the FR/NFR boundary is not crisp: an item can carry both aspects (Dalpiaz et al., 2019), and backlog items mix several requirements in one entry (van Can & Dalpiaz, 2025). Third, label-free approaches are possible but still well behind supervised ones (Alhoshan et al., 2023; Luo et al., 2022). Our own results sharpen this picture. For the coarse FR/NFR decision a linear TF-IDF model was as good as DistilBERT, whereas for separating quality classes the transformer was clearly better. For a team, this suggests that a transformer pays off mainly at Level 2 of the framework, and that a cheap linear model may be enough at Level 1. The open question is how well any of these models works on genuine backlog text, where items are terse, informal and full of project jargon.

B. Implications for Backlog Management

For a Scrum team the value lies less in the label itself than in the conversation it starts. Picture a team refining twenty new stories before sprint planning. The classifier tags fourteen with high confidence. It flags “As an administrator, I want to see a log of all login attempts” as functional with a security concern, and it sends three ambiguous items to the review queue. The Product Owner confirms the security tag, so the team adds retention and access rules to the acceptance criteria before the work starts, not after a later audit. Used this way, the classifier supports the Definition of Done and leaves responsibility for ordering and scope with the Product Owner (Schwaber & Sutherland, 2020). It also complements story-quality checks such as QUS (Lucassen et al., 2016a): one tool asks whether a story is well formed, the other asks what kind of concern it raises.

C. Challenges

Table IV summarises the main deployment challenges and how the framework addresses each one. Three need further comment. *Imbalance* is structural: some quality classes will always be rare, and a macro-averaged score is the honest way to report it (Johnson & Khoshgoftaar, 2019). *Format shift* is the largest unknown: models trained on “The system shall…” sentences meet “As a ⟨role⟩…” stories, bug reports and one-line chores in a real tracker (van Can & Dalpiaz, 2025). *Label drift* follows from the backlog being an informal and evolving model of the work (Sedano et al., 2019): a team's notion of “operability” may change as its product matures.

TABLE IV
Deployment challenges, supporting evidence and mitigations.

Challenge	Evidence	Mitigation in the framework
Class imbalance (PO: 1 item; FT: 10)	(Johnson & Khoshgoftaar, 2019)	Weighted loss (Eq. 1); macro-F1; merge rare classes
FR/NFR ambiguity, mixed items	(Dalpiaz et al., 2019; van Can & Dalpiaz, 2025)	F+Q label; split multi-requirement items
SRS-to-story format shift	(Lucassen et al., 2016b; van Can & Dalpiaz, 2025)	Parse role/goal/benefit; fine-tune on team data
Over-confident predictions	Our Section III-D	Confidence gate; audit sample; calibration check
Leakage and optimistic splits	(Hey et al., 2020b)	Project- or time-based evaluation before roll-out
Scarce labelled data	(Alhoshan et al., 2023; Tunstall et al., 2022)	Zero-/few-shot start; active labelling in review
Compute and privacy limits	(Sanh et al., 2019)	Distilled model on-premise; no external API

D. Threats to Validity

Construct validity. PROMISE labels were assigned by students around 2007 (Cleland-Huang, Settimi, et al., 2007), and the FR/NFR boundary itself is contested (Dalpiaz et al., 2019). A model that agrees with these labels has learned one annotation convention, not a ground truth. *Internal validity.* We used one random seed per fold, no hyper-parameter search, and a partially frozen encoder. A different configuration could change the Task A ranking, where the difference was not significant, and could narrow or widen the Task B gap. The baselines and DistilBERT did, however, share identical folds and pre-processing. *External validity.* Random 10-fold splits put requirements from the same project into both training and test data, which inflates all scores; Hey et al. (2020b) quantified a drop of about five points when folds were made properly project-specific. PROMISE contains English “shall” statements, so our results say nothing direct about user stories or other languages. *Conclusion validity.* With 249 items in Task B, each test fold holds about 25 items, so fold-level accuracies vary considerably. We therefore report pooled scores and paired McNemar tests rather than claiming small differences. Finally, the literature synthesis is not a systematic review, and relevant studies may have been missed.

E. Ethical and Practical Considerations

Backlogs can contain customer names, contract details and security weaknesses. Sending them to an external API may breach confidentiality, which is a practical argument for small models that run on-premise. Automation bias is the other risk: if a high-confidence label is never questioned, the team may stop thinking about quality at all. Keeping the label visible but editable, and auditing a sample every release, limits that risk.

VI. Conclusion and Future Work

We set out to judge whether BERT-family classifiers can support agile backlog management. The literature shows steady progress on PROMISE-style text, from indicator terms to NoRBERT, but also a habit of reporting near-ceiling scores

under protocols that cannot be compared. Our own small replication found that a CPU-trained DistilBERT was no better than a linear SVM at separating functional from non-functional requirements (macro-F1 0.901 vs. 0.916), but clearly better at telling the four most frequent quality classes apart (0.919 vs. 0.832). Random folds flatter both models, so these figures are upper estimates for unseen projects. We therefore propose using such models as refinement assistants: they surface quality concerns early and route uncertain items to a person, while ordering and scope remain with the Product Owner.

Four steps would make the evidence stronger. (1) Build an openly licensed corpus of real user stories and backlog items with F/Q aspect labels, so that results stop depending on PROMISE. (2) Evaluate with project-held-out and time-ordered splits as standard. (3) Compare fully fine-tuned BERT and RoBERTa with few-shot SetFit and zero-shot baselines under one protocol, using label budgets a real team could afford. (4) Run a field study in which a team uses the confidence-gated classifier during refinement, and measure whether quality issues are found earlier.

References

Alhoshan, W., Ferrari, A., & Zhao, L. (2023). Zero-shot learning for requirements classification: An exploratory study. *Information and Software Technology, 159*, Article 107202. https://doi.org/10.1016/j.infsof.2023.107202

Alsanoosy, T. (2026). Requirement classification using transfer learning models. *Arabian Journal for Science and Engineering*, *51*(15), 18047–18064. https://doi.org/10.1007/s13369-026-11242-w

Binkhonain, M., & Zhao, L. (2019). A review of machine learning algorithms for identification and classification of non-functional requirements. *Expert Systems with Applications: X, 1*, Article 100001. https://doi.org/10.1016/j.eswax.2019.100001

Choetkiertikul, M., Dam, H. K., Tran, T., Pham, T., Ghose, A., & Menzies, T. (2019). A deep learning model for estimating story points. *IEEE Transactions on Software Engineering, 45*(7), 637–656. https://doi.org/10.1109/TSE.2018.2792473

Cleland-Huang, J., Mazrouee, S., Huang, L., & Port, D. (2007). *nfr* [Data set]. Zenodo. https://doi.org/10.5281/zenodo.268542

Cleland-Huang, J., Settimi, R., Zou, X., & Solc, P. (2007). Automated classification of non-functional requirements. *Requirements Engineering, 12*(2), 103–120. https://doi.org/10.1007/s00766-007-0045-1

Dalpiaz, F., Dell'Anna, D., Aydemir, F. B., & Çevikol, S. (2019). Requirements classification with interpretable machine learning and dependency parsing. In *2019 IEEE 27th International Requirements Engineering Conference (RE)* (pp. 142–152). IEEE. https://doi.org/10.1109/RE.2019.00025

Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)* (pp. 4171–4186). Association for Computational Linguistics. https://doi.org/10.18653/v1/N19-1423

Eltahier, S., Dawood, O., & Saeed, I. (2025). BERT fine-tuning for software requirement classification: Impact of model components and dataset size. *Information, 16*(11), Article 981. https://doi.org/10.3390/info16110981

Hey, T., Keim, J., Koziolek, A., & Tichy, W. F. (2020a). NoRBERT: Transfer learning for requirements classification. In *2020 IEEE 28th International Requirements Engineering Conference (RE)* (pp. 169–179). IEEE. https://doi.org/10.1109/RE48521.2020.00028

Hey, T., Keim, J., Koziolek, A., & Tichy, W. F. (2020b). *Supplementary material of "NoRBERT: Transfer learning for requirements classification"* [Computer software]. Zenodo. https://doi.org/10.5281/zenodo.8347866

Hou, X., Zhao, Y., Liu, Y., Yang, Z., Wang, K., Li, L., Luo, X., Lo, D., Grundy, J., & Wang, H. (2024). Large language models for software engineering: A systematic literature review. *ACM Transactions on Software Engineering and Methodology, 33*(8), 1–79. https://doi.org/10.1145/3695988

Hujainah, F., Bakar, R. B. A., Abdulgabber, M. A., & Zamli, K. Z. (2018). Software requirements prioritisation: A systematic literature review on significance, stakeholders, techniques and challenges. *IEEE Access, 6*, 71497–71523. https://doi.org/10.1109/ACCESS.2018.2881755

Johnson, J. M., & Khoshgoftaar, T. M. (2019). Survey on deep learning with class imbalance. *Journal of Big Data, 6*(1), Article 27. https://doi.org/10.1186/s40537-019-0192-5

Kici, D., Malik, G., Cevik, M., Parikh, D., & Başar, A. (2021). A BERT-based transfer learning approach to text classification on software requirements specifications. In *Proceedings of the Canadian Conference on Artificial Intelligence*. Canadian Artificial Intelligence Association. https://doi.org/10.21428/594757db.a4880a62

Kurtanović, Z., & Maalej, W. (2017). Automatically classifying functional and non-functional requirements using supervised machine learning. In *2017 IEEE 25th International Requirements Engineering Conference (RE)* (pp. 490–495). IEEE. https://doi.org/10.1109/RE.2017.82

Lima, M., Valle, V., Costa, E., Lira, F., & Gadelha, B. (2019). Software engineering repositories: Expanding the PROMISE database. In *Proceedings of the XXXIII Brazilian Symposium on Software Engineering* (pp. 427–436). ACM. https://doi.org/10.1145/3350768.3350776

Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., & Stoyanov, V. (2019). *RoBERTa: A robustly optimized BERT pretraining approach* (arXiv:1907.11692). arXiv. https://doi.org/10.48550/arXiv.1907.11692

Lucassen, G., Dalpiaz, F., van der Werf, J. M. E. M., & Brinkkemper, S. (2016a). Improving agile requirements: The Quality User Story framework and tool. *Requirements Engineering, 21*(3), 383–403. https://doi.org/10.1007/s00766-016-0250-x

Lucassen, G., Dalpiaz, F., van der Werf, J. M. E. M., & Brinkkemper, S. (2016b). The use and effectiveness of user stories in practice. In *Requirements Engineering: Foundation for Software Quality* (pp. 205–222). Springer. https://doi.org/10.1007/978-3-319-30282-9_14

Luo, X., Xue, Y., Xing, Z., & Sun, J. (2022). PRCBERT: Prompt learning for requirement classification using BERT-based pretrained language models. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering* (pp. 1–13). ACM. https://doi.org/10.1145/3551349.3560417

Mejía-Domínguez, F. A., Palacio, R. R., Borrego, G., González-López, S., & Del-Puerto-Flores, J. A. (2026). LOONA: An LLM assistant for Spanish language user story quality assurance in agile development. *Mathematics, 14*(19), Article 3637. https://doi.org/10.3390/math14193637

Navarro-Almanza, R., Juárez-Ramírez, R., & Licea, G. (2017). Towards supporting software engineering using deep learning: A case of software requirements classification. In *2017 5th International Conference in Software Engineering Research and Innovation (CONISOFT)* (pp. 116–120). IEEE. https://doi.org/10.1109/CONISOFT.2017.00021

Rahimi, N., Eassa, F., & Elrefaei, L. (2021). One- and two-phase software requirement classification using ensemble deep learning. *Entropy, 23*(10), Article 1264. https://doi.org/10.3390/e23101264

Rahman, M. A., Haque, M. A., Tawhid, M. N. A., & Siddik, M. S. (2019). Classifying non-functional requirements using RNN variants for quality software development. In *Proceedings of the 3rd ACM SIGSOFT International Workshop on Machine Learning Techniques for Software Quality Evaluation* (pp. 25–30). ACM. https://doi.org/10.1145/3340482.3342745

Sanh, V., Debut, L., Chaumond, J., & Wolf, T. (2019). *DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter* (arXiv:1910.01108). arXiv. https://doi.org/10.48550/arXiv.1910.01108

Schwaber, K., & Sutherland, J. (2020). *The Scrum guide*. https://scrumguides.org/scrum-guide.html

Sedano, T., Ralph, P., & Péraire, C. (2019). The product backlog. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)* (pp. 200–211). IEEE. https://doi.org/10.1109/ICSE.2019.00036

Sun, C., Qiu, X., Xu, Y., & Huang, X. (2019). How to fine-tune BERT for text classification? In *Chinese Computational Linguistics* (pp. 194–206). Springer. https://doi.org/10.1007/978-3-030-32381-3_16

Tikayat Ray, A., Cole, B. F., Pinon Fischer, O. J., White, R. T., & Mavris, D. N. (2023). aeroBERT-Classifier: Classification of aerospace requirements using BERT. *Aerospace, 10*(3), Article 279. https://doi.org/10.3390/aerospace10030279

Tunstall, L., Reimers, N., Jo, U. E. S., Bates, L., Korat, D., Wasserblat, M., & Pereg, O. (2022). *Efficient few-shot learning without prompts* (arXiv:2209.11055). arXiv. https://doi.org/10.48550/arXiv.2209.11055

van Can, A. T., & Dalpiaz, F. (2025). Locating requirements in backlog items: Content analysis and experiments with large language models. *Information and Software Technology, 179*, Article 107644. https://doi.org/10.1016/j.infsof.2024.107644

Winkler, J., & Vogelsang, A. (2016). Automatic classification of requirements based on convolutional neural networks. In *2016 IEEE 24th International Requirements Engineering Conference Workshops (REW)* (pp. 39–45). IEEE. https://doi.org/10.1109/REW.2016.021

Yucalar, F. (2023). Developing an advanced software requirements classification model using BERT: An empirical evaluation study on newly generated Turkish data. *Applied Sciences, 13*(20), Article 11127. https://doi.org/10.3390/app132011127

Zhao, L., Alhoshan, W., Ferrari, A., Letsholo, K. J., Ajagbe, M. A., Chioasca, E.-V., & Batista-Navarro, R. T. (2022). Natural language processing for requirements engineering: A systematic mapping study. *ACM Computing Surveys, 54*(3), 1–41. https://doi.org/10.1145/3444689